

DOCUMENTACIÓN DEL MÓDULO DE ENTIDADES FLEXIBLES DE OMNIMERGE

1. Nombre del módulo

Módulo de creación, personalización y organización de entidades

Este módulo constituye el núcleo inicial de OmniMerge. Su propósito es permitir que cada usuario pueda crear entidades de cualquier naturaleza, definir libremente sus atributos, establecer valores disponibles y asignar uno o varios valores a cada entidad.

Las entidades no estarán limitadas a personajes humanos o criaturas. Una entidad puede representar cualquier elemento que el usuario desee modelar.

Ejemplos:

- Personajes.
- Animales.
- Criaturas.
- Objetos.
- Vehículos.
- Países.
- Planetas.
- Colores.
- Elementos naturales.
- Conceptos abstractos.
- Equipos.
- Organizaciones.
- Productos.
- Lugares.
- Cualquier otro elemento definido por el usuario.

2. Descripción general

El módulo permitirá que cada usuario construya su propia biblioteca de entidades.

Cada entidad podrá tener:

- Nombre.
- Código.
- Descripción.
- Imagen.
- Tipo de entidad.
- Estado.
- Etiquetas.
- Colecciones.
- Atributos personalizados.
- Uno o varios valores por atributo.
- Grupos visuales de atributos.
- Orden personalizado de presentación.

Por ejemplo, un usuario puede crear la entidad:

Nombre: Pyron

Tipo: Dragón

Descripción: Dragón legendario perteneciente al Reino de Astralis.

Después puede asignarle atributos:

Elemento:

- Fuego
- Luz

Habilidades:

- Volar
- Regeneración
- Aliento de fuego

Poder:

- 95

Personalidad:

- Orgulloso
- Leal
- Impulsivo

El atributo **Elemento** puede tener muchas opciones disponibles:

Fuego

Agua

Aire

Tierra

Luz

Oscuridad

Hielo

Electricidad

Sin embargo, Pyron solamente utiliza:

Fuego

Luz

Por tanto, debe existir una diferencia clara entre:

1. El atributo.
2. Las opciones disponibles para ese atributo.
3. El atributo asignado a una entidad.
4. Los valores concretos que la entidad utiliza.

3. Objetivo general del módulo

Desarrollar un módulo flexible que permita a los usuarios crear, personalizar, organizar y reutilizar entidades mediante atributos y valores dinámicos, sin depender de estructuras rígidas establecidas previamente por el sistema.

4. Objetivos específicos

- Permitir que cada usuario cree entidades de cualquier tipo.
- Permitir que los usuarios definan sus propios tipos de entidad.
- Permitir la creación de atributos personalizados.
- Permitir atributos numéricos, textuales, booleanos, fechas, colores, opciones y otros tipos de datos.
- Permitir que un atributo tenga opciones previamente definidas.
- Permitir que una entidad tenga uno o varios valores para un mismo atributo.
- Permitir valores libres además de opciones predefinidas.
- Organizar entidades en múltiples colecciones.
- Organizar atributos en múltiples grupos visuales.
- Definir el orden de presentación de los atributos.
- Reutilizar atributos y opciones en múltiples entidades.
- Facilitar filtros, búsquedas, comparaciones y futuras simulaciones.

5. Principios del diseño

5.1. Flexibilidad

El sistema no debe asumir que todas las entidades tienen los mismos atributos.

Ejemplo:

Dragón:

- Poder
- Elemento
- Capacidad de vuelo

Vehículo:

- Velocidad máxima
- Marca
- Tipo de combustible

Color:

- Código hexadecimal
- Saturación
- Luminosidad

Cada usuario podrá definir los atributos que necesite.

5.2. Reutilización

Un atributo se crea una sola vez y puede utilizarse en muchas entidades.

Ejemplo:

Atributo: Poder

Entidades que lo utilizan:

- Pyron

- Aquaris
- Terron
- Voltrax

No será necesario volver a crear el atributo **Poder** para cada entidad.

5.3. Multiplicidad

Un atributo podrá aceptar:

- Un único valor.
- Varios valores.

Ejemplo de valor único:

Rareza:

- Legendario

Ejemplo de múltiples valores:

Habilidades:

- Volar
- Teletransportarse
- Regenerarse

5.4. Propiedad por usuario

Las entidades, atributos, opciones, colecciones y grupos pertenecerán al usuario que los creó. Esto permitirá que diferentes usuarios mantengan bibliotecas independientes.

Ejemplo:

Usuario A:

- Poder mágico
- Elemento
- Reino

Usuario B:

- Cilindrada
- Velocidad máxima
- Tipo de combustible

5.5. Organización visual

Los atributos podrán agruparse para facilitar la presentación de una entidad.

Ejemplo:

Información general

- Edad
- Origen
- Descripción

Características físicas

- Altura
- Peso
- Color

Combate

- Poder
- Velocidad
- Resistencia

Personalidad

- Valentía
- Lealtad
- Agresividad

Un atributo podrá pertenecer a más de un grupo.

6. Estructura general de la base de datos

El módulo utilizará las siguientes tablas:

users

entity_types
entities

attributes
attribute_options

entity_attributes
entity_attribute_values

collections
collection_entity

attribute_groups
attribute_group_attribute

tags
entity_tag

Las tablas esenciales para la primera versión son:

users
entity_types
entities
attributes

attribute_options
entity_attributes
entity_attribute_values
collections
collection_entity
attribute_groups
attribute_group_attribute

Las tablas de etiquetas pueden incorporarse después.

7. Tabla **users**

7.1. Propósito

Almacena los usuarios registrados en OmniMerge.

Cada usuario será propietario de sus entidades, atributos, tipos, colecciones y grupos.

7.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK, autoincremental	Identificador del usuario
name	VARCHAR(100)	NOT NULL	Nombre completo
username	VARCHAR(50)	UNIQUE, NOT NULL	Nombre de usuario
email	VARCHAR(150)	UNIQUE, NOT NULL	Correo electrónico
password	VARCHAR(255)	NOT NULL	Contraseña cifrada
role	VARCHAR(20)	NOT NULL	Rol del usuario
status	VARCHAR(20)	NOT NULL	Estado de la cuenta
last_login_at	TIMESTAMP	NULL	Último inicio de sesión
email_verified_at	TIMESTAMP	NULL	Fecha de verificación
remember_token	VARCHAR(100)	NULL	Token de sesión
created_at	TIMESTAMP	NULL	Fecha de creación
updated_at	TIMESTAMP	NULL	Fecha de actualización
deleted_at	TIMESTAMP	NULL	Borrado lógico

7.3. Valores recomendados

role

ADMIN

USER

status

ACTIVE

INACTIVE

SUSPENDED

7.4. Ejemplo

id: 1

name: Grover Romeo

username: grverlearner

email: usuario@ejemplo.com

role: USER

status: ACTIVE

8. Tabla **entity_types**

8.1. Propósito

Permite que el usuario defina categorías generales para sus entidades.

Ejemplos:

- Personaje.
- Dragón.
- Animal.
- Vehículo.
- Planeta.
- Color.
- Objeto.
- Concepto.

El tipo no limita completamente a la entidad, pero sirve para clasificarla y sugerir atributos.

8.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK	Identificador
user_id	BIGINT	FK, NOT NULL	Usuario propietario
code	VARCHAR(30)	NOT NULL	Código interno
name	VARCHAR(100)	NOT NULL	Nombre del tipo
description	TEXT	NULL	Descripción
icon	VARCHAR(100)	NULL	Icono
color	VARCHAR(20)	NULL	Color visual
status	VARCHAR(20)	NOT NULL	Estado
created_at	TIMESTAMP	NULL	Creación
updated_at	TIMESTAMP	NULL	Actualización
deleted_at	TIMESTAMP	NULL	Borrado lógico

8.3. Restricción recomendada

UNIQUE(user_id, code)

Esto permite que diferentes usuarios usen el mismo código, pero evita duplicados dentro de una misma cuenta.

8.4. Ejemplo

id: 1

user_id: 1

code: DRAGON

name: Dragón

description: Criaturas mágicas utilizadas en universos de fantasía.

color: #D62828

status: ACTIVE

9. Tabla **entities**

9.1. Propósito

Almacena las entidades creadas por los usuarios.

Una entidad puede representar cualquier elemento.

9.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK	Identificador
user_id	BIGINT	FK, NOT NULL	Usuario propietario
entity_type_id	BIGINT	FK, NULL	Tipo de entidad
code	VARCHAR(30)	NOT NULL	Código interno
name	VARCHAR(150)	NOT NULL	Nombre
slug	VARCHAR(180)	NOT NULL	Identificador amigable
description	TEXT	NULL	Descripción
image	VARCHAR(255)	NULL	Ruta de imagen
status	VARCHAR(20)	NOT NULL	Estado
visibility	VARCHAR(20)	NOT NULL	Visibilidad
created_at	TIMESTAMP	NULL	Fecha de creación
updated_at	TIMESTAMP	NULL	Fecha de actualización
deleted_at	TIMESTAMP	NULL	Borrado lógico

9.3. Valores recomendados

status

ACTIVE

INACTIVE

ARCHIVED

visibility

PRIVATE

PUBLIC

UNLISTED

9.4. Restricciones

UNIQUE(user_id, code)

UNIQUE(user_id, slug)

9.5. Ejemplo

id: 10

user_id: 1

entity_type_id: 1

code: ENT-PYRON

name: Pyron

slug: pyron

description: Dragón legendario de fuego.

image: entities/pyron.webp

status: ACTIVE

visibility: PRIVATE

10. Tabla **attributes**

10.1. Propósito

Almacena los atributos personalizados creados por los usuarios.

Un atributo representa una característica que puede asignarse a una entidad.

Ejemplos:

- Poder.
- Elemento.
- Edad.
- Peso.
- Habilidades.
- Personalidad.
- Fecha de aparición.
- Puede volar.
- Color principal.

10.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK	Identificador
user_id	BIGINT	FK, NOT NULL	Usuario propietario
code	VARCHAR(30)	NOT NULL	Código

name	VARCHAR(100)	NOT NULL	Nombre
slug	VARCHAR(120)	NOT NULL	Identificador amigable
description	TEXT	NULL	Descripción
data_type	VARCHAR(30)	NOT NULL	Tipo de dato
allows_multiple	BOOLEAN	NOT NULL	Permite varios valores
uses_options	BOOLEAN	NOT NULL	Usa opciones predefinidas
allows_custom_values	BOOLEAN	NOT NULL	Permite valores libres
is_required	BOOLEAN	NOT NULL	Es obligatorio
is_filterable	BOOLEAN	NOT NULL	Puede utilizarse en filtros
is_comparable	BOOLEAN	NOT NULL	Puede utilizarse en comparaciones
min_numeric_value	DECIMAL(15,4)	NULL	Valor mínimo
max_numeric_value	DECIMAL(15,4)	NULL	Valor máximo
min_length	INT	NULL	Longitud mínima
max_length	INT	NULL	Longitud máxima
default_value	JSON	NULL	Valor predeterminado
unit	VARCHAR(30)	NULL	Unidad
icon	VARCHAR(100)	NULL	Icono
color	VARCHAR(20)	NULL	Color
status	VARCHAR(20)	NOT NULL	Estado
created_at	TIMESTAMP	NULL	Creación
updated_at	TIMESTAMP	NULL	Actualización
deleted_at	TIMESTAMP	NULL	Borrado lógico

10.3. Tipos de datos

TEXT

Texto corto.

Ejemplo:

Alias: El Señor de la Llama

LONG_TEXT

Texto extenso.

Ejemplo:

Historia: Pyron nació en las montañas volcánicas...

INTEGER

Número entero.

Ejemplo:

Edad: 540

DECIMAL

Número decimal.

Ejemplo:

Altura: 12.75

BOOLEAN

Valor verdadero o falso.

Ejemplo:

Puede volar: Sí

DATE

Fecha.

Ejemplo:

Fecha de creación: 2026-08-03

DATETIME

Fecha y hora.

COLOR

Color.

Ejemplo:

Color principal: #FF4500

OPTION

Una o varias opciones predefinidas.

Ejemplo:

Elemento:

- Fuego

- Agua

- Tierra

URL

Dirección web.

IMAGE

Imagen asociada.

ENTITY_REFERENCE

Referencia a otra entidad.

Ejemplo:

Maestro: Drakon

JSON

Estructura personalizada avanzada.

No es necesario implementar todos los tipos desde la primera versión.

10.4. Tipos recomendados para el MVP

TEXT

LONG_TEXT

INTEGER

DECIMAL

BOOLEAN

DATE

COLOR

OPTION

10.5. Ejemplo de atributo de opción múltiple

id: 1

user_id: 1

code: ELEMENT

name: Elemento

data_type: OPTION

allows_multiple: true

uses_options: true

allows_custom_values: false

is_required: false

is_filterable: true

is_comparable: true

status: ACTIVE

10.6. Ejemplo de atributo numérico

id: 2

user_id: 1

code: POWER

name: Poder

data_type: INTEGER

allows_multiple: false

uses_options: false

allows_custom_values: true

min_numeric_value: 0

max_numeric_value: 100

unit: puntos

11. Tabla **attribute_options**

11.1. Propósito

Almacena los valores predefinidos disponibles para un atributo.

Ejemplo:

Atributo: Elemento

Opciones disponibles:

- Fuego
- Agua
- Aire
- Tierra
- Luz
- Oscuridad

La existencia de una opción no significa que todas las entidades deban utilizarla.

11.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK	Identificador
attribute_id	BIGINT	FK, NOT NULL	Atributo propietario
code	VARCHAR(30)	NOT NULL	Código
name	VARCHAR(100)	NOT NULL	Nombre
description	TEXT	NULL	Descripción
image	VARCHAR(255)	NULL	Imagen
color	VARCHAR(20)	NULL	Color
numeric_weight	DECIMAL(15,4)	NULL	Peso opcional
sort_order	INT	NOT NULL	Orden
status	VARCHAR(20)	NOT NULL	Estado
created_at	TIMESTAMP	NULL	Creación
updated_at	TIMESTAMP	NULL	Actualización
deleted_at	TIMESTAMP	NULL	Borrado lógico

11.3. Restricción

UNIQUE(attribute_id, code)

11.4. Ejemplo

id: 1

attribute_id: 1

code: FIRE

name: Fuego
description: Afinidad con energía térmica y llamas.
color: #FF4500
sort_order: 1
status: ACTIVE

id: 2
attribute_id: 1
code: WATER
name: Agua
color: #008CFF
sort_order: 2
status: ACTIVE

12. Tabla **entity_attributes**

12.1. Propósito

Representa la asignación de un atributo a una entidad.

Esta tabla indica que una entidad utiliza un atributo, incluso aunque todavía no tenga valores registrados.

La relación es:

Entidad



Atributo asignado



Uno o varios valores

12.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK	Identificador
entity_id	BIGINT	FK, NOT NULL	Entidad
attribute_id	BIGINT	FK, NOT NULL	Atributo
custom_label	VARCHAR(100)	NULL	Nombre personalizado
is_visible	BOOLEAN	NOT NULL	Se muestra en la vista
is_featured	BOOLEAN	NOT NULL	Atributo destacado
sort_order	INT	NOT NULL	Orden
notes	TEXT	NULL	Notas

created_at	TIMESTAMP	NULL	Creación
updated_at	TIMESTAMP	NULL	Actualización

12.3. Restricción

UNIQUE(entity_id, attribute_id)

Esto evita que el mismo atributo sea asignado dos veces a la misma entidad.

Los múltiples valores se almacenarán en **entity_attribute_values**.

12.4. Ejemplo

id: 100

entity_id: 10

attribute_id: 1

custom_label: Afinidades elementales

is_visible: true

is_featured: true

sort_order: 1

Esto significa:

Pyron utiliza el atributo Elemento.

Todavía no indica qué elementos tiene. Esa información se almacena en la siguiente tabla.

13. Tabla **entity_attribute_values**

13.1. Propósito

Almacena los valores concretos que una entidad posee para un atributo.

Es la tabla central del sistema dinámico.

Una entidad puede tener:

- Un valor.
- Varios valores.
- Una opción predefinida.
- Un valor escrito libremente.
- Un valor numérico.
- Un valor booleano.
- Una fecha.
- Una referencia a otra entidad.

13.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK	Identificador
entity_attribute_id	BIGINT	FK, NOT NULL	Atributo asignado
attribute_option_id	BIGINT	FK, NULL	Opción seleccionada

text_value	TEXT	NULL	Valor textual
integer_value	BIGINT	NULL	Número entero
decimal_value	DECIMAL(15,4)	NULL	Número decimal
boolean_value	BOOLEAN	NULL	Valor lógico
date_value	DATE	NULL	Fecha
datetime_value	DATETIME	NULL	Fecha y hora
color_value	VARCHAR(20)	NULL	Color
url_value	VARCHAR(500)	NULL	URL
image_value	VARCHAR(255)	NULL	Imagen
referenced_entity_id	BIGINT	FK, NULL	Otra entidad
json_value	JSON	NULL	Valor complejo
custom_value	VARCHAR(255)	NULL	Valor libre adicional
sort_order	INT	NOT NULL	Orden
created_at	TIMESTAMP	NULL	Creación
updated_at	TIMESTAMP	NULL	Actualización

13.3. Regla principal

Aunque la tabla contiene varios campos de valor, cada registro utilizará únicamente el campo correspondiente al tipo del atributo.

Ejemplo numérico:

Atributo: Poder

integer_value: 95

Ejemplo textual:

Atributo: Alias

text_value: El Señor de la Llama

Ejemplo opción:

Atributo: Elemento

attribute_option_id: Fuego

13.4. Ejemplo de múltiples valores

Pyron tiene:

Elemento:

- Fuego

- Luz

En **entity_attributes**:

id: 100

entity_id: 10

attribute_id: 1

En **entity_attribute_values**:

Registro 1:

entity_attribute_id: 100

attribute_option_id: 1

sort_order: 1

Registro 2:

entity_attribute_id: 100

attribute_option_id: 5

sort_order: 2

13.5. Ejemplo numérico

Atributo asignado:

entity_id: Pyron

attribute_id: Poder

Valor:

integer_value: 95

13.6. Ejemplo booleano

Atributo: Puede volar

boolean_value: true

13.7. Ejemplo de valor libre

Atributo:

Título honorífico

Valor:

custom_value: Guardián de la Montaña Roja

14. Tabla **collections**

14.1. Propósito

Permite organizar entidades en colecciones reutilizables.

Una colección puede representar:

- Categoría temática.
- Proyecto.

- Universo futuro.
- Lista de favoritos.
- Grupo de selección.
- Biblioteca.
- Clasificación personalizada.

Ejemplos:

Dragones legendarios

Personajes de Astralis

Criaturas de fuego

Favoritos

Participantes futuros

14.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK	Identificador
user_id	BIGINT	FK, NOT NULL	Propietario
code	VARCHAR(30)	NOT NULL	Código
name	VARCHAR(120)	NOT NULL	Nombre
description	TEXT	NULL	Descripción
image	VARCHAR(255)	NULL	Imagen
icon	VARCHAR(100)	NULL	Icono
color	VARCHAR(20)	NULL	Color
visibility	VARCHAR(20)	NOT NULL	Visibilidad
status	VARCHAR(20)	NOT NULL	Estado
sort_order	INT	NOT NULL	Orden
created_at	TIMESTAMP	NULL	Creación
updated_at	TIMESTAMP	NULL	Actualización
deleted_at	TIMESTAMP	NULL	Borrado lógico

14.3. Ejemplo

id: 5

user_id: 1

code: FIRE_CREATURES

name: Criaturas de fuego

description: Entidades relacionadas con fuego, calor o volcanes.

color: #E63946
status: ACTIVE

15. Tabla **collection_entity**

15.1. Propósito

Relaciona entidades con colecciones.

Una entidad puede pertenecer a muchas colecciones.

Una colección puede contener muchas entidades.

La relación es muchos a muchos:

Entities N:M Collections

15.2. Campos

Campo	Tipo	Restricción	Descripción
collection_id	BIGINT	FK, PK	Colección
entity_id	BIGINT	FK, PK	Entidad
sort_order	INT	NOT NULL	Orden
added_at	TIMESTAMP	NULL	Fecha de incorporación
notes	TEXT	NULL	Notas

15.3. Clave primaria

PRIMARY KEY(collection_id, entity_id)

15.4. Ejemplo

Pyron pertenece a:

Dragones

Criaturas de fuego

Legendarios

Favoritos

Esto genera cuatro registros en **collection_entity**.

16. Tabla **attribute_groups**

16.1. Propósito

Permite agrupar atributos para mejorar la organización y presentación visual.

Ejemplos:

Información general

Características físicas

Combate

Personalidad

Historia

16.2. Campos

Campo	Tipo	Restricción	Descripción
id	BIGINT	PK	Identificador
user_id	BIGINT	FK, NOT NULL	Usuario propietario
code	VARCHAR(30)	NOT NULL	Código
name	VARCHAR(100)	NOT NULL	Nombre
description	TEXT	NULL	Descripción
icon	VARCHAR(100)	NULL	Icono
color	VARCHAR(20)	NULL	Color
layout_type	VARCHAR(30)	NOT NULL	Presentación
collapsible	BOOLEAN	NOT NULL	Puede contraerse
default_expanded	BOOLEAN	NOT NULL	Abierto inicialmente
sort_order	INT	NOT NULL	Orden
status	VARCHAR(20)	NOT NULL	Estado
created_at	TIMESTAMP	NULL	Creación
updated_at	TIMESTAMP	NULL	Actualización
deleted_at	TIMESTAMP	NULL	Borrado lógico

16.3. Tipos de diseño

LIST

GRID

CARDS

TABLE

COMPACT

16.4. Ejemplo

id: 1

user_id: 1

code: COMBAT

name: Combate

description: Estadísticas relacionadas con enfrentamientos.

icon: sword
color: #B5172F
layout_type: GRID
collapsible: true
default_expanded: true
sort_order: 3

17. Tabla **attribute_group_attribute**

17.1. Propósito

Relaciona atributos con grupos de atributos.

Un grupo puede contener muchos atributos.

Un atributo puede aparecer en muchos grupos.

Ejemplo:

Velocidad

puede aparecer en:

Características físicas

Combate

Estadísticas principales

17.2. Campos

Campo	Tipo	Restricción	Descripción
attribute_group_id	BIGINT	FK, PK	Grupo
attribute_id	BIGINT	FK, PK	Atributo
sort_order	INT	NOT NULL	Orden
custom_label	VARCHAR(100)	NULL	Etiqueta dentro del grupo
is_featured	BOOLEAN	NOT NULL	Destacado

17.3. Clave primaria

PRIMARY KEY(attribute_group_id, attribute_id)

18. Tabla opcional **entity_type_attribute**

18.1. Propósito

Permite sugerir atributos según el tipo de entidad.

No obliga a que todas las entidades de un tipo tengan los mismos atributos.

Ejemplo:

Tipo de entidad: Dragón

Atributos sugeridos:

- Elemento
- Poder
- Capacidad de vuelo
- Tamaño

Una entidad de tipo Dragón podrá:

- Utilizar esos atributos.
- Omitir algunos.
- Agregar otros atributos personalizados.

18.2. Campos

Campo	Tipo	Restricción	Descripción
entity_type_id	BIGINT	FK, PK	Tipo
attribute_id	BIGINT	FK, PK	Atributo sugerido
is_required	BOOLEAN	NOT NULL	Obligatorio para el tipo
default_sort_order	INT	NOT NULL	Orden predeterminado
default_group_id	BIGINT	FK, NULL	Grupo recomendado

Esta tabla es recomendable, pero puede incorporarse después de completar el núcleo.

19. Tablas opcionales de etiquetas

19.1. Tabla **tags**

Permite crear etiquetas rápidas.

Ejemplos:

Legendario

Fuego

Favorito

Pendiente

Jefe final

Campos:

Campo	Tipo	Descripción
id	BIGINT	Identificador
user_id	BIGINT	Propietario
name	VARCHAR(50)	Nombre
color	VARCHAR(20)	Color
created_at	TIMESTAMP	Creación

updated_at TIMESTAMP Actualización

19.2. Tabla **entity_tag**

Relaciona etiquetas y entidades.

PRIMARY KEY(entity_id, tag_id)

Las etiquetas sirven para clasificaciones rápidas, mientras que las colecciones representan agrupaciones más estructuradas.

20. Diagrama textual de relaciones

USERS

- |— 1:N ENTITY_TYPES
- |— 1:N ENTITIES
- |— 1:N ATTRIBUTES
- |— 1:N COLLECTIONS
- |— 1:N ATTRIBUTE_GROUPS

ENTITY_TYPES

- |— 1:N ENTITIES

ENTITIES

- |— 1:N ENTITY_ATTRIBUTES
- |— N:M COLLECTIONS
- |— N:M TAGS

ATTRIBUTES

- |— 1:N ATTRIBUTE_OPTIONS
- |— 1:N ENTITY_ATTRIBUTES
- |— N:M ATTRIBUTE_GROUPS
- |— N:M ENTITY_TYPES

ENTITY_ATTRIBUTES

- |— 1:N ENTITY_ATTRIBUTE_VALUES

ATTRIBUTE_OPTIONS

- |— 1:N ENTITY_ATTRIBUTE_VALUES

21. Relaciones explicadas

21.1. Usuario y entidad

Un usuario puede crear muchas entidades.

Una entidad pertenece a un usuario.

Relación:

users 1:N entities

21.2. Usuario y atributo

Un usuario puede crear muchos atributos.

Un atributo pertenece a un usuario.

Relación:

users 1:N attributes

21.3. Entidad y atributo

Una entidad puede tener muchos atributos.

Un atributo puede utilizarse en muchas entidades.

La relación se resuelve mediante:

entity_attributes

Relación conceptual:

entities N:M attributes

21.4. Atributo y opción

Un atributo puede tener muchas opciones disponibles.

Una opción pertenece a un atributo.

Relación:

attributes 1:N attribute_options

21.5. Atributo asignado y valor

Un atributo asignado a una entidad puede tener uno o varios valores.

Relación:

entity_attributes 1:N entity_attribute_values

21.6. Entidad y colección

Una entidad puede pertenecer a muchas colecciones.

Una colección puede contener muchas entidades.

Relación:

entities N:M collections

21.7. Atributo y grupo

Un atributo puede estar en muchos grupos.

Un grupo puede contener muchos atributos.

Relación:
attributes N:M attribute_groups

22. Ejemplo completo de funcionamiento

22.1. Usuario

Usuario: Grover

22.2. Tipo de entidad

Tipo: Dragón

22.3. Entidad

Nombre: Pyron

Código: ENT-PYRON

Tipo: Dragón

Descripción: Dragón legendario nacido en un volcán.

22.4. Atributos creados

Elemento

Poder

Habilidades

Puede volar

Edad

Color principal

Historia

22.5. Opciones del atributo Elemento

Fuego

Agua

Tierra

Aire

Luz

Oscuridad

22.6. Opciones del atributo Habilidades

Volar

Regeneración

Aliento de fuego

Telepatía

Control del clima

22.7. Atributos asignados a Pyron

Elemento

Poder

Habilidades
Puede volar
Edad
Color principal
Historia

22.8. Valores concretos

Elemento:

- Fuego
- Luz

Poder:

- 95

Habilidades:

- Volar
- Regeneración
- Aliento de fuego

Puede volar:

- Sí

Edad:

- 540

Color principal:

- #C1121F

Historia:

- Pyron nació en las montañas volcánicas del Reino de Astralis.

23. Ejemplo de registros simplificados

entities

10 | Pyron | Dragón legendario

attributes

- 1 | Elemento | OPTION | múltiple
- 2 | Poder | INTEGER | único
- 3 | Habilidades | OPTION | múltiple
- 4 | Puede volar | BOOLEAN | único

attribute_options

- 1 | Elemento | Fuego

2 | Elemento | Agua
3 | Elemento | Tierra
4 | Elemento | Luz
5 | Habilidades | Volar
6 | Habilidades | Regeneración
7 | Habilidades | Aliento de fuego

entity_attributes

100 | Pyron | Elemento
101 | Pyron | Poder
102 | Pyron | Habilidades
103 | Pyron | Puede volar

entity_attribute_values

1000 | entity_attribute=100 | option=Fuego
1001 | entity_attribute=100 | option=Luz
1002 | entity_attribute=101 | integer=95
1003 | entity_attribute=102 | option=Volar
1004 | entity_attribute=102 | option=Regeneración
1005 | entity_attribute=102 | option=Aliento de fuego
1006 | entity_attribute=103 | boolean=true

24. Reglas de negocio

24.1. Propiedad

Un usuario solamente podrá modificar:

- Sus propias entidades.
- Sus propios atributos.
- Sus propias colecciones.
- Sus propios grupos.
- Sus propias opciones.

24.2. Asignación de atributos

Una entidad no podrá tener dos veces el mismo atributo dentro de **entity_attributes**.

Los múltiples valores se guardarán en **entity_attribute_values**.

24.3. Validación por tipo

El valor deberá coincidir con **data_type**.

Ejemplos:

- **INTEGER** acepta números enteros.
- **DECIMAL** acepta números decimales.
- **BOOLEAN** acepta verdadero o falso.
- **DATE** acepta fechas.
- **OPTION** acepta opciones del atributo.

- **COLOR** acepta un código de color válido.
-

24.4. Valores múltiples

Si:

`allows_multiple = false`

solo podrá existir un valor activo.

Si:

`allows_multiple = true`

podrán existir varios valores.

24.5. Opciones

Si:

`uses_options = true`

el usuario deberá seleccionar opciones de **attribute_options**.

Si además:

`allows_custom_values = true`

podrá agregar valores personalizados.

24.6. Límites numéricos

Si un atributo tiene:

`min_numeric_value = 0`

`max_numeric_value = 100`

el sistema rechazará valores fuera de ese rango.

24.7. Eliminación de atributos

No se recomienda eliminar físicamente un atributo que ya está siendo utilizado.

Debe emplearse borrado lógico mediante:

`deleted_at`

o cambiar su estado a:

INACTIVE

24.8. Eliminación de opciones

Una opción utilizada por entidades no debería eliminarse físicamente.

Puede marcarse como inactiva para impedir nuevas asignaciones sin perder el historial.

24.9. Orden

El usuario podrá definir:

- Orden de grupos.
 - Orden de atributos.
 - Orden de valores.
 - Orden de entidades dentro de colecciones.
-

25. Vista de detalle de una entidad

La vista podría organizarse así:

PYRON

Dragón legendario del Reino de Astralis

[Información general]

Tipo: Dragón

Edad: 540 años

Estado: Activo

[Características físicas]

Color principal: Rojo oscuro

Altura: 12.5 metros

[Combate]

Poder: 95

Resistencia: 88

Velocidad: 73

[Elementos]

- Fuego

- Luz

[Habilidades]

- Volar

- Regeneración

- Aliento de fuego

[Personalidad]

- Orgullosa

- Leal

- Impulsivo

[Colecciones]

- Dragones

- Criaturas de fuego

- Entidades legendarias

26. Formulario para crear una entidad

26.1. Datos generales

Nombre: [_____]

Código: [_____]

Tipo:

[Seleccionar tipo ▼]

Descripción:

[_____]

[_____]

Imagen:

[Seleccionar archivo]

Estado:

[Activo ▼]

26.2. Atributos

[+ Agregar atributo]

Al seleccionar un atributo:

Atributo:

[Elemento ▼]

Valores:

☒ Fuego

☐ Agua

☐ Tierra

☒ Luz

[Guardar]

Para un atributo numérico:

Atributo:

[Poder ▼]

Valor:

[95]

Rango permitido:

0 – 100

Para texto múltiple:

Atributo:

[Alias ▼]

Valores:

[Señor de la Llama] [x]

[Guardián del Volcán] [x]

[+ Agregar valor]

27. Formulario para crear atributos

Nombre:

[Elemento]

Código:

[ELEMENT]

Descripción:

[Tipo de energía o naturaleza elemental]

Tipo de dato:

[Opción ▼]

¿Permite múltiples valores?

[Sí]

¿Utiliza opciones predefinidas?

[Sí]

¿Permite valores personalizados?

[No]

¿Se puede utilizar en filtros?

[Sí]

¿Se puede utilizar en comparaciones?

[Sí]

¿Es obligatorio?

[No]

Después:

Opciones disponibles:

1. Fuego
2. Agua
3. Tierra
4. Aire
5. Luz
6. Oscuridad

[+ Agregar opción]

28. Formas de visualizar las entidades

28.1. Vista de tarjetas

Cada entidad se presenta con:

- Imagen.
- Nombre.
- Tipo.
- Atributos destacados.
- Colecciones.
- Estado.

28.2. Vista de tabla

Columnas configurables:

Nombre

Tipo

Poder

Elemento

Rareza

Estado

28.3. Vista de cuadrícula

Útil para entidades visuales.

28.4. Vista detallada

Muestra todos los grupos y atributos.

28.5. Vista comparativa

Permite comparar varias entidades:

Atributo	Pyron	Aquaris	Terron
Poder	95	82	90
Velocidad	73	91	60
Elemento	Fuego, Luz	Agua	Tierra

29. Filtros posibles

Gracias a esta estructura se podrán implementar filtros como:
Mostrar entidades cuyo Poder sea mayor que 80.

Mostrar entidades que tengan el elemento Fuego.

Mostrar entidades que puedan volar.

Mostrar entidades pertenecientes a Dragones.

Mostrar entidades con las habilidades Volar y Regeneración.

Mostrar entidades pertenecientes a dos colecciones específicas.

30. Orden recomendado de implementación

Etapla 1: usuarios

users

Etapla 2: tipos y entidades

entity_types

entities

Etapla 3: atributos

attributes

attribute_options

Etapla 4: asignación de atributos

entity_attributes

entity_attribute_values

Etapla 5: colecciones

collections

collection_entity

Etapla 6: grupos visuales

attribute_groups

attribute_group_attribute

Etapla 7: tipos sugeridos

entity_type_attribute

Etapla 8: etiquetas

tags

entity_tag

31. Migraciones Laravel necesarias

Se recomienda crear las migraciones en este orden:

1. create_users_table
2. create_entity_types_table
3. create_entities_table
4. create_attributes_table
5. create_attribute_options_table
6. create_entity_attributes_table
7. create_entity_attribute_values_table
8. create_collections_table
9. create_collection_entity_table
10. create_attribute_groups_table
11. create_attribute_group_attribute_table
12. create_entity_type_attribute_table
13. create_tags_table
14. create_entity_tag_table

El orden es importante porque las tablas dependientes requieren que las tablas principales ya existan.

32. Modelos Laravel

Los modelos recomendados son:

User
EntityType
Entity
Attribute
AttributeOption
EntityAttribute
EntityAttributeValue
Collection
AttributeGroup
Tag

Las tablas intermedias pueden manejarse mediante relaciones **belongsToMany**.

33. Relaciones Eloquent esperadas

User

hasMany(EntityType)
hasMany(Entity)
hasMany(Attribute)
hasMany(Collection)
hasMany(AttributeGroup)

Entity

belongsTo(User)
belongsTo(EntityType)
hasMany(EntityAttribute)
belongsToMany(Collection)
belongsToMany(Tag)

Attribute

belongsTo(User)
hasMany(AttributeOption)
hasMany(EntityAttribute)
belongsToMany(AttributeGroup)
belongsToMany(EntityType)

EntityAttribute

belongsTo(Entity)
belongsTo(Attribute)
hasMany(EntityAttributeValue)

EntityAttributeValue

belongsTo(EntityAttribute)
belongsTo(AttributeOption)
belongsTo(ReferencedEntity)

Collection

belongsTo(User)
belongsToMany(Entity)

AttributeGroup

belongsTo(User)
belongsToMany(Attribute)

34. Diferencia con el diseño original

El diseño original contenía:

personaje
atributo
personaje_atributo
calificativo
atributo_calificativo
personaje_atributo_calificativo

El diseño actualizado utiliza:

entities
attributes
entity_attributes
attribute_options
entity_attribute_values

Correspondencia:

Diseño original	Diseño propuesto
personaje	entities
atributo	attributes
personaje_atributo	entity_attributes
calificativo	attribute_options
atributo_calificativo	attribute_options.attribute_id
personaje_atributo_calificativo	entity_attribute_values

La relación **atributo_calificativo** deja de necesitar una tabla independiente porque cada opción contiene directamente su **attribute_id**.

La tabla **entity_attribute_values** permite guardar tanto opciones como valores libres, numéricos, booleanos y fechas.

35. Ventajas del modelo propuesto

- Permite cualquier tipo de entidad.
- Permite atributos creados por usuarios.
- Permite atributos reutilizables.
- Permite uno o varios valores.
- Permite opciones predefinidas.
- Permite valores libres.
- Permite números, texto, fechas y booleanos.
- Permite agrupación visual.
- Permite muchas colecciones por entidad.
- Facilita búsquedas y filtros.
- Facilita comparaciones.
- Facilita futuras simulaciones.
- Evita crear columnas diferentes para cada atributo.
- Mantiene el historial lógico mediante estados y borrado lógico.
- Se adapta correctamente a Laravel y Eloquent.
- Puede crecer sin modificar constantemente la tabla **entities**.

36. Conclusión

El módulo de entidades flexibles será la base principal de OmniMerge.

El sistema no utilizará una estructura rígida donde todas las entidades tengan columnas como poder, velocidad, color o edad. En su lugar, cada usuario podrá construir sus propios atributos y asignarlos libremente.

La estructura central será:

Entidad



Atributo asignado



Uno o varios valores

Los atributos podrán tener opciones disponibles:

Atributo



Muchas opciones

Pero cada entidad solamente seleccionará las opciones que le correspondan.

Además:

Una entidad podrá pertenecer a muchas colecciones.

Un atributo podrá pertenecer a muchos grupos.

Un atributo podrá utilizarse en muchas entidades.

Una entidad podrá tener muchos atributos.

Un atributo asignado podrá tener muchos valores.

Esta estructura permitirá que OmniMerge sea una plataforma altamente creativa, reutilizable y adaptable, preparada para incorporar posteriormente universos, temporadas, torneos, simulaciones, rankings y narrativas emergentes.